

Оптимизация

Передача данных Host / Device

- Является дорогостоящей операцией
- Возможна асинхронная передача

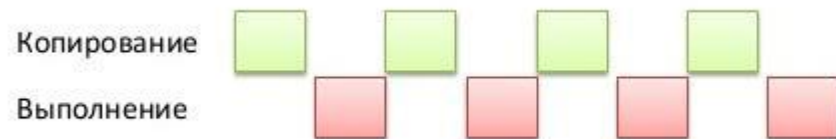
`cudaMemcpy()`

`cudaMemcpyAsync()`

- Синхронная передача



- Асинхронная передача



Типы памяти устройства

- Streaming Multiprocessor
 - Регистровая память
 - Разделяемая память
 - Константная память
- DDRAM
 - Локальная память
 - Глобальная память
- Текстурный блок
 - Текстурная память

Объединенное чтение DRAM

- Выравнивание исходных данных по границе слова
- Потоки warp-а должны осуществлять одновременное чтение DDRAM

Разделяемая память и конфликты

- Общая для всех потоков блока
- Распределяется между блоками
- Разбивается на банки (32-битные слова)

Float vs Double

- Арифметика с float числами быстрее чем с double числами

Использовать “f” при объявлении числовых констант
(3.14f)

Умножение / степень

- По возможности использовать оператор сдвига
- Для известных целых значений степеней использовать явное перемножение вместо вызова `pow()`

Аппаратные аналоги функций

- `__sinf()` — `sinf()`
- `__cosf()` — `cosf()`
- `__expf()` — `expf()`
- ...

Операторы ветвления

- Операторы управления потоком команд (if, switch, for, while, do-while) отрицательно сказываются на производительности
- Следить за исполнением внутри варпа, в идеале все нити идут по одному пути, иначе возможно последовательное выполнение

Процесс оптимизации

- Определить что ограничивает производительность:
 - Пропускная способность памяти
 - Исполнение инструкций
 - Задержка
 - Комбинации
- Исследовать ограничители в порядке важности
 - Насколько эффективно работает
 - Анализ и нахождение проблем
 - Применить оптимизации

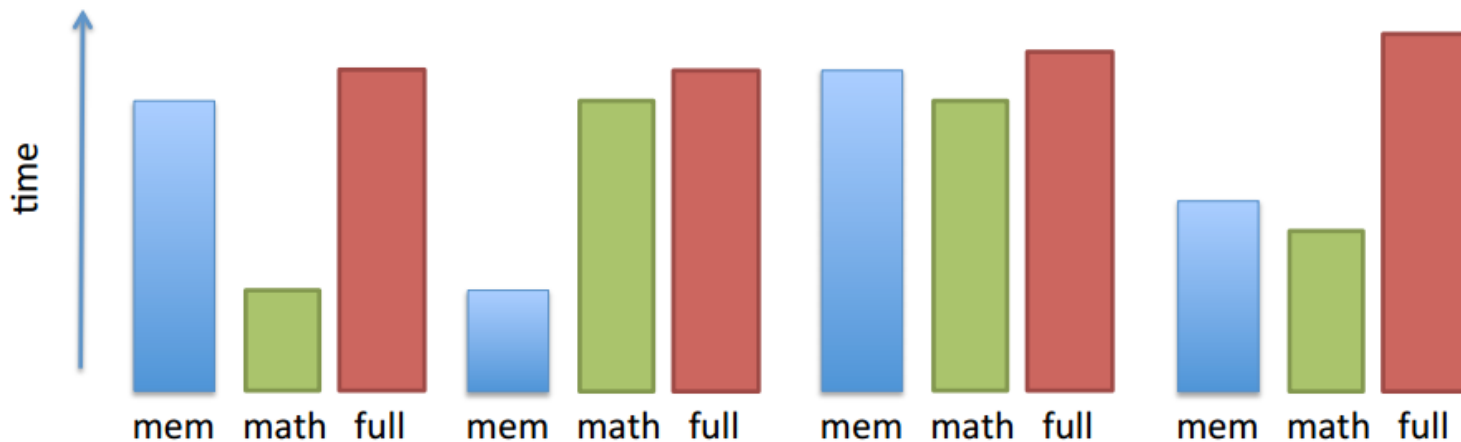
Анализ через модификацию кода

- Измеряем время работы 2-х модификаций:
 - Минимум инструкций, только доступ к памяти(mem)
 - Минимум обращений к памяти, только инструкции(math)

Удобно для алгоритмов не использующих зависимость по данным для ветвления или адресации.

- Сравниваем время работы модифицированных ядер
 - Помогает определить во что мы упираемся
 - Насколько хорошо идет перекрытие арифметических операций и чтений из памяти

Анализ через модификацию кода



Ограничитель -
память

Ограничитель -
инструкции

Баланс

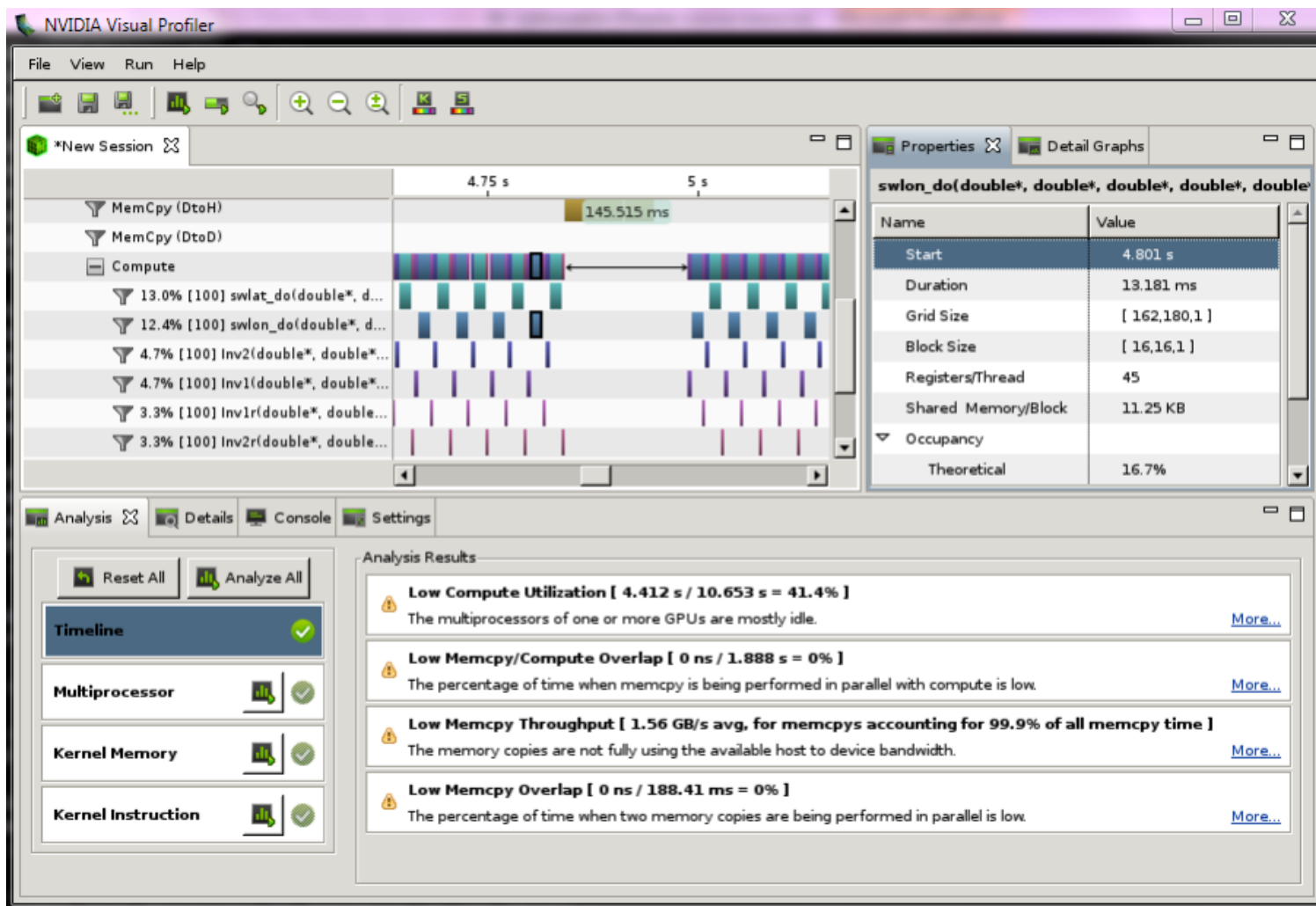
- Хорошее перекрытие математики и памяти: нет проблем с задержками

- Хорошее перекрытие математики и памяти: нет проблем с задержками

- Хорошее перекрытие математики и памяти: нет проблем с задержками

- Плохое перекрытие математики и памяти: нет проблем с задержками

Анализ через модификацию кода



Анализ через модификацию кода

The screenshot displays the Visual Studio IDE with the 'Night CUDA Samples' project open. The main window shows the source code for 'matrixMul_kernel.cu'. A debugger window is active, showing the 'Process' tab with the thread 'Thread [13188816]'. A 'CUDA (4.0.0, 04.0.0)' window is also visible. A 'NVIDIA Parallel Night - CUDA Focus Picker' dialog box is open, showing the 'Block' and 'Thread' dimensions. The 'Block' is set to 4, 0, 0 and the 'Thread' is set to 14, 0, 0. The 'Examples' section shows the block index 129 for coordinates 10, 0 and 10, 5 for coordinates 10, 5. The 'Memory' window at the bottom shows a list of memory addresses and their corresponding values. The 'Locals' window on the right shows the values of local variables: 'blockIdx' (x=4, y=0, z=0), 'blockDim' (x=16, y=16, z=1), 'gridDim' (x=8, y=5, z=1), 'a' (??), 'b' (??), 'bx' (4), 'by' (0), 'tx' (14), 'ty' (0), 'aBegin' (0), 'aEnd' (47), 'aStep' (16), 'bBegin' (64), 'bStep' (2048), 'Cub' (0), 'c' (??), 'C' (0x00120000), 'A' (0x00120000), 'B' (0x00113000), 'uA' (48), and 'uB' (128).

```
// Step size used to iterate through the sub-matrices of A
int aStep = BLOCK_SIZE;

// Index of the first sub-matrix of B processed by the block
int bBegin = BLOCK_SIZE * bx;

// Step size used to iterate through the sub-matrices of B
int bStep = BLOCK_SIZE * by;

// Cub is used to store the element of the block sub-matrix
// that is computed by the thread
float Cub = 0;

// Loop over all the sub-matrices of A and B
// required to compute the block sub-matrix
for (int a = aBegin, b = bBegin;
     a <= aEnd;
     a += aStep, b += bStep) {

    // Declaration of the shared memory array A
    // store the sub-matrix of A
    __shared__ float A[BLOCK_SIZE][BLOCK_SIZE];
```

Block: 4, 0, 0 Dimensions: 8, 5, 1
Thread: 14, 0, 0 Dimensions: 16, 16, 1

Examples
#129 for block index 129
10 for coordinates 10, 0
10, 5 for coordinates 10, 5

Address	Value
0x00113000	0.17999917
0x00113008	0.89681491
0x00113010	0.10892056
0x00113018	0.71721548
0x00113020	0.27188939
0x00113028	0.32767726
0x00113030	0.70702201
0x00113038	0.32767726
0x00113040	0.70702201
0x00113048	0.11164515
0x00113050	0.34851474
0x00113058	0.34851474
0x00113060	0.74705223
0x00113068	0.13171728
0x00113070	0.21298867
0x00113078	0.60927163
0x00113080	0.88696287
0x00113088	0.94161739
0x00113090	0.31685290
0x00113098	0.98294014
0x001130A0	0.72765181
0x001130A8	0.28931946
0x001130B0	0.30326244
0x001130B8	0.0097964415

Name	Value	Type
blockIdx	{x = 4, y = 0, z = 0}	const int
blockDim	{x = 16, y = 16, z = 1}	const dim
gridDim	{x = 8, y = 5, z = 1}	const dim
a	??	int
b	??	int
bx	4	int
by	0	int
tx	14	int
ty	0	int
aBegin	0	int
aEnd	47	int
aStep	16	int
bBegin	64	int
bStep	2048	int
Cub	0	float
c	??	int
C	0x00120000	__device__
A	0x00120000 0.20208646	__device__
B	0x00113000 0.80645152	__device__
uA	48	__shared__
uB	128	__shared__

Счетчики профайлера

- timestamp
- gld_32b / gst_32b
- gridsize
- gld_64b / gst_62b
- threadblocksize
- gld_128b / gst_128b
- dynsmemperblock
- gld_request
- stasmemperblock
- gst_incoherent
- regperthread
- gst_coherent
- memtransferdir
- gst_request
- memtransfersize
- local load
- Streamid
- local store
- gputime
- branch
- cputime
- divergent_branch
- Occupancy
- instructions
- gld_incoherent
- warp_serialize
- gld_coherent
- cta_launched

- Значение имеют не цифры, а их приращение и соотношение
- Для ядер нужно стремиться чтобы стремились к нулю непоследовательные обращения к памяти

(gld_incoherent, gld_coherent, gst_incoherent, gst_coherent)

Арифметика - Память

- Оптимальное соотношение для Tesla C2050 инструкции : байты:
 - $\sim 3.6 : 1$, float при включенном ECC
 - $\sim 4.5 : 1$, float при выключенном ECC

Счетчик профилировщика (на варп)

- Сколько инструкций исполнилось
 - Instructions executed
- Конфликты банков разделяемой памяти
 - l1_shared_bank_conflict,
 - shared_load,
 - shared_store
- Конфликты существенны если:
 - $l1_shared_bank_conflict \gg (shared_load + shared_store)$
 - $l1_shared_bank_conflict \gg instructions_issued$

Спиллинг регистров

- Достигнут предел доступных регистров, компилятор начинает использовать локальную память
 - Дополнительная нагрузка шины памяти
 - Дополнительные инструкции
 - Не всегда проблема

Счетчики `l1_local_load_hit`, `l1_local_load_miss`

CUDA Occupancy calculator

Just follow steps 1, 2, and 3 below! (or click here for help)	
1.) Select Compute Capability (click):	
1,2	
2.) Enter your resource usage:	
Threads / Warp	Threads Per Block
Warps / Multiprocessor	Registers Per Thread
Threads / Multiprocessor	Shared Memory Per Block (bytes)
Thread Blocks / Multiprocessor	
Total # of 32-bit registers / Multiprocessor	
Shared Memory / Multiprocessor (bytes)	
(Don't edit anything below this line)	
3.) GPU Occupancy Data is displayed here and in the graphs:	
Allocation Per Thread Block	Active Threads per Multiprocessor
Warps	Active Warps per Multiprocessor
Registers	Active Thread Blocks per Multiprocessor
Shared Memory	Occupancy of each Multiprocessor
These data are used in computing the occupancy	
Maximum Thread Blocks Per Multiprocessor	Blocks
Limited by Max Warps / Multiprocessor	4
Limited by Registers / Multiprocessor	3
Limited by Shared Memory / Multiprocessor	32
Thread Block Limit Per Multiprocessor highlighted	RED

Оптимизация